

PyInstrument: Final Report

By: Jose Francisco Palomino Lau, Michael Ogunwale, Ursula Parker

Project Overview

What Is This Project & What Does It Do?

Pyinstrument is a statistical profiling tool for Python that assists developers in identifying performance bottlenecks within their code. Rather than tracing every individual function call, it samples the call stack at regular intervals, which minimizes runtime overhead while providing a clear and accurate representation of where the program spends its execution time. This approach makes it easier to pinpoint inefficiencies and optimize overall code performance. Additionally, Pyinstrument generates clear, easy-to-read reports that include interactive visualizations to support performance analysis and debugging.

Highlights of Each Testing Stage

Unit Testing: Coverage Expansion

The unit testing effort focused on validating the core functionality and reliability of key components within the PyInstrument package, primarily the Profiler and Frame classes. Tests covered common profiling workflows and main API paths to ensure correct behavior under typical usage. For the Profiler class, our tests verified proper initialization, start and stop behavior, session resetting, context manager support, and output generation with duration information; these all reflect standard developer interactions with the tool.

Additional edge case tests evaluated robustness under boundary conditions. These confirmed that the profiler handles empty sample sets without crashing, raises appropriate errors for invalid arguments, fails gracefully when loading corrupted session files, enforces a `RuntimeError` on repeated `stop()` calls, and returns `None` for empty session frames. Together, these tests strengthened reliability in error handling and empty-state behavior.

The Frame class was also tested to ensure correct parent-child relationships within the profiling call tree. These tests validated child addition, reference integrity, array lengths, string tagging, and handling of multiple children, confirming accurate internal structure and execution hierarchy representation.

The most meaningful result was increased confidence in both common and edge-case scenarios. Overall coverage remained largely stable, increasing slightly from 74% to 75%, as the new tests reinforced existing code paths rather than expanding into new ones. No critical

defects were identified; however, we documented the coverage improvements, highlighting potential areas for future testing.

Unit Testing: Mocking & Stubbing

Mocking and stubbing were used to test components that depend on external behavior, allowing our unit tests to focus on interaction correctness rather than underlying implementations. Tests targeted profiler-related functions that interact with external storage, profiling execution, and report rendering.

Additional mocked tests focused on the performance profiling workflow involving computationally intensive tasks. The profiler was replaced with a mock using `unittest.mock.patch`, enabling validation of control flow without executing real profiling. A heavy task was used to reflect realistic usage scenarios where incorrect profiling behavior could impact data accuracy.

Mocking was also applied to the HTML rendering component to evaluate report generation under minimal data conditions. By replacing the report saving function with a mock, the tests confirmed that HTML reports could be produced successfully without relying on file system operations or full datasets.

Mutation Testing

Mutation testing was conducted using `mutmut` in order to evaluate the strength and effectiveness of the existing unit test suite by introducing small, systematic changes to the codebase. The tool was configured to mutate covered lines within the `PyInstrument` package, while excluding non-critical files (such as entry points, vendor code, and rendering components). This setup ensured that our mutation testing only focused on meaningful application logic rather than infrastructure or boilerplate code.

The most meaningful result of this process was the identification of weakly tested logic through the surviving mutants. Initially, 376 mutations survived, indicating areas where tests did not sufficiently detect behavioral changes. After adding some targeted tests, the number of surviving mutants was reduced to 373. While the numerical improvement was modest, the results clearly highlighted specific logic paths that lacked adequate test coverage, providing actionable insight into test suite weaknesses.

Based on this analysis, additional tests were added to address these weak spots. For example, there was a flaw where mutating a list return value to `None` had no observable effect; so, we introduced a new test to enforce that the function cannot return `None`, successfully killing the mutant and strengthening test coverage for that logic. Overall, mutation testing proved valuable in revealing limitations of the existing tests and guiding focused improvements to test quality rather than surface-level coverage metrics.

Static Analysis & Code Smells

Static analysis was performed using Ruff in order to identify code smells, risky patterns, and maintainability issues within the project. The analysis was executed across the codebase using ruff check, revealing 77 issues, including style violations, unused variables, unsafe exception handling, and readability concerns. These findings highlighted areas of the code that, while functionally correct, posed risks to long-term maintainability and clarity.

The most meaningful result of the analysis was the identification of sections of the codebase that were difficult to maintain or understand. Notable issues included lambda functions assigned to variables, bare except clauses that could mask runtime errors, unused local variables, and equality comparisons against boolean values. Collectively, these patterns reduced code readability, obscured program intent, and increased the likelihood of subtle bugs during future development or refactoring.

Several issues were fixed as part of this analysis to improve code quality and safety. Lambda expressions were refactored into named functions using def, bare exception handlers were replaced with more specific exceptions, unused variables were removed, and boolean equality checks were rewritten using idiomatic truth checks. We resolved the simple issues automatically using ruff check. --fix, while others required manual intervention. After these changes, the total number of reported issues was reduced to 35. Overall, the static analysis process improved code clarity and robustness while providing a documented baseline for future quality improvements.

Integration Testing

Integration testing was conducted to evaluate how the major components of the PyInstrument system work together with their dependencies, focusing on the interactions between the Profiler core module and multiple output-rendering modules. Tests were implemented to validate synchronous profiling workflows involving the text renderer, HTML renderer, and JSON renderer. Each test verified that profiling data captured during execution could be correctly passed between components and that it was all rendered correctly.

The most meaningful result of these integration tests was the identification and validation of more complex behaviors that are not detectable through unit testing alone. By running the profiler end-to-end with real workloads, the tests confirmed that stopping and starting worked properly, data was captured accurately, and that outputs were formatted correctly (in plain text, HTML markup, and JSON structures).

No functional defects were identified during integration testing; however, we did document some critical setup assumptions and execution requirements (such as environment configuration and renderer expectations). Overall, the integration tests strengthened confidence in the system's ability to operate correctly as a cohesive whole.

System Testing

System testing was conducted to validate PyInstrument's end-to-end functionality from a real user's perspective: using the tool through its command-line interface. These tests focused on full profiling workflows, including executing Python scripts, collecting runtime performance data, and generating profiling reports in both HTML and JSON formats. By treating the system as a black box, these tests ensured that core functionality behaved correctly without reliance on internal implementation details.

The most meaningful result of system testing was confirming that the primary profiling workflow functions correctly on most supported environments. We demonstrated that PyInstrument can execute user scripts, collect profiling samples, and generate valid output reports (that included function call hierarchies, execution times, and required metadata fields).

However, the system testing phase did reveal a crucial environment-specific limitation that our unit and integration tests could not detect. In macOS environments, PyInstrument silently failed to generate output files when using the `-o` flag, even though profiling completed successfully and the correct results were displayed in the terminal. This behavior was reproducible across multiple attempts and renderer options, suggesting a potential compatibility or environmental issue related to Apple Silicon, internal file-writing logic, or macOS directory protections. While no system crash occurred, the silent failure represented a critical usability concern.

All these findings were documented as known limitations and edge cases. Overall, our system testing strengthened confidence in pyinstrument's core end-to-end workflow, while highlighting some significant real-world constraints that were only observable at the system level.

Security Testing

Security testing was conducted using the Bandit static analysis tool to examine Python source files within the PyInstrument directory for common security vulnerabilities and unsafe coding patterns. The scan focused on Python-specific risks, including the use of assert statements in production code, which should be removed in optimized execution modes, and the use of `exec`, which may introduce arbitrary code execution vulnerabilities. All core components of the project were included in the analysis, such as the CLI entry point, profiling logic, renderers, context managers, and supporting utilities.

The most meaningful result of the security scan was the absence of any high or critical severity vulnerabilities in the codebase. There were a total of 19 issues reported, consisting of 17 low-severity findings (related to assert usage) and 2 medium-severity findings (related to `exec`). While these issues do not indicate immediate exploitation risks, they do highlight areas where code reliability, safety, and defensive programming practices could be improved upon.

We did not fix any vulnerabilities as part of this testing phase; instead, all of the identified issues were documented along with recommended remediation strategies. These included replacing

assert statements with explicit runtime checks and exceptions, and substituting `exec` with safer alternatives such as `runpy` or controlled execution mechanisms.

Performance Testing

We conducted performance testing in order to evaluate the behavior of the PyInstrument profiler under increasing load and stress conditions using the Locust load testing framework. These tests targeted a Flask-based `/profile` endpoint that executes profiling routines, simulating realistic, concurrent user behavior. Our load tests assessed system performance under moderate, sustained traffic; our stress and spike tests examined system stability and responsiveness as concurrency increased toward types of saturation.

The most meaningful result of the performance testing phase was observing how profiling overhead impacts response times as user load increases, while still maintaining overall system stability. Under a load of 10 concurrent users, the system sustained approximately 40 requests per second with a median response time of 13 ms and no failures. CPU utilization remained within acceptable bounds, indicating that profiling operations introduced manageable overhead without degrading responsiveness. These results demonstrate that PyInstrument performs efficiently in typical usage scenarios.

Stress testing further showed that the system remained decently robust even under significantly higher concurrency; however, it could be improved. With up to 50 concurrent users, the endpoint sustained over 230 requests per second with zero failures, but there was a higher level of latency (with some requests taking up to 37ms). However, response times remained relatively stable, indicating that performance was predictable and there was no resource contention or bottlenecks.

Only mild performance-related defects were identified during testing. Overall, our performance testing confirmed that PyInstrument is capable of handling both moderate and heavy profiling workloads with consistent and reliable performance, however, some latency should be expected at higher loads.

Most Significant Issues Found

Profiler Overhead Under Higher Load

The impact of profiling overhead under higher concurrency was revealed through performance testing, specifically load, stress, and spike tests conducted with Locust against the PyInstrument-instrumented endpoint. These tests showed that as the number of concurrent users increased, response times grew in part due to the overhead introduced by the profiler itself rather than the underlying application logic.

This finding matters because profiler overhead can distort performance measurements and obscure real application bottlenecks. If not accounted for, developers may misinterpret results and make inaccurate optimization decisions. The issue was not fixed, as profiling overhead is an inherent characteristic of runtime instrumentation, but it was explicitly documented so that performance results are interpreted with this limitation in mind.

The impact on system quality is primarily on the reliability of performance metrics. By identifying and documenting this behavior, the testing process improves the accuracy and usefulness of performance analysis, ensuring that PyInstrument's results are used appropriately when evaluating and optimizing system performance.

Fragility With Dependencies

The fragility arising from external dependencies was primarily revealed through unit tests in the suite that we developed for the class, where inconsistent Python versions or missing optional packages caused intermittent test failures across different environments. These tests, profiling simple synthetic functions like related loops and short CPU bound calls, exposed how subtle mismatches such as varying NumPy installations or pip resolver behaviors altered profiling outputs or prevented the suite from running entirely.

This issue matters because unreliable dependencies undermine the reproducibility of performance measurements, turning objective profiler data into environment-specific noise that undermines trust in results for academic validation or collaborative development. Without addressing it, the project risks producing misleading insights that vary by machine, complicating comparisons and debugging.

While a full fix remains pending due to PyInstrument's broad compatibility needs, the team documented precise setup steps including pinned dependency versions and virtual environment instructions in the repository docs to mitigate recurrence. This documentation reduces setup friction but doesn't eliminate the risk entirely. Consequently, system quality improves in controlled setups yet retains moderate fragility for diverse deployments, highlighting a key area for future pinning or containerization efforts.

Sensitivity to Build Environments

Unit tests in the courseProjectDocs suite were the ones that clearly revealed this sensitivity to build environments. These tests, which profile simple functions like repeated loops and short CPU tasks, would fail or give different results depending on the machine. For example, one computer with Python 3.10 and specified NumPy version would pass all tests perfectly but switching to Python 3.11 or a different pip setup would cause some tests to crash or show wrong profiling times.

This problem matters a lot because it makes performance results unreliable. You can't trust what PyInstrument says if the numbers change just because someone uses a different computer or

OS. For school projects or teamwork, this means you can't easily repeat experiments or compare results between people which defeats the whole point of having tests and documentation.

We couldn't completely fix it, since PyInstrument needs to work on many setups, but we documented very clear steps in the repository. This includes creating a virtual environment with venv, installing exact packages and running tests in clean folders. We also added a requirements.txt file and setup script to make it easier. These changes would help new users get started faster and reduce errors.

Even with this documentation, the system still has some sensitivity for very different environments like Windows vs Linux or Python versions. The overall quality is much better now for standard setups, but it remains a weak point for production use or diverse teams. The next step could be using Docker containers to lock everything down completely, making tests run exactly the same everywhere.

File Creation Issues on Silicon Mac

When running pyinstrument on an Apple Silicon Mac, the `-o` flag fails to generate an output file, despite the profiler executing successfully and printing a full profiling summary to the terminal. This behavior represents a silent failure, as no error or warning was displayed even though the expected output file was not created. Because exporting profiling results is a core feature of the tool, this issue significantly impacts usability for affected users, particularly those relying on file-based reports for analysis or sharing.

This defect was uncovered through system testing, specifically during an end-to-end CLI test case that simulated real-world usage by executing a Python script via the PyInstrument command-line interface with the `-o` output flag. The test verified the expected file generation, which failed. This failure was consistently reproduced across multiple runs, different output file names, and alternative renderers such as JSON.

There are a few theories that could explain this behavior. One possibility is an Apple Silicon compatibility issue, where lower-level system calls used by PyInstrument behave differently on ARM-based architectures, potentially affecting how profiling data is flushed or written to disk. Another theory is an internal PyInstrument bug in which the tool falls back to printing results to the terminal when file output fails, without raising an exception or warning. A third possibility involves macOS directory protection, where file-writing operations may be silently restricted in certain locations (such as iCloud-synced or protected directories), preventing output files from being created.

The issue was not fixed during this project due to its environment-specific nature and the need for deeper investigation beyond the scope of testing. Instead, it was documented in detail, including reproduction steps, observed behavior, and plausible causes. Hopefully, future

developers will be able to fix this issue and make Pyinstrument fully usable on Silicon Macs, as advertised.

Improvements made to the project

The project introduced several substantial improvements that strengthened the reliability, transparency and analytical depth of the PyInstrument profiling environment. The main objective was to better understand how the profiler behaves under realistic workloads and to make the testing suite easier to execute and interpret across different setups. By refining both the test design and the documentation, the repository now provides a more reliable foundation for profiling based performance evaluation.

A key improvement was the clarification and standardization of the environment setup process. The installation and configuration instructions were revised to ensure that the test suite ran consistently across systems with different Python versions and dependency settings. This change allowed the team to replicate results without encountering compatibility issues or misconfigurations that previously affected test reproducibility. Alongside this, we validated the existing test structure, expanded coverage where needed, and improved these descriptions to make each scenario clearer in terms of its purpose and expected behavior.

To extend the project's validation scope, several new test cases were added under the `courseProjectDocs` directory to exercise PyInstrument in more realistic and repeatable scenarios. These tests covered small CPU bound functions, repeated function calls and simple loops that ran under different input sizes, allowing the profiler to be triggered multiple times over controlled pieces of code. By running these scenarios systematically, the project now exposes how PyInstrument behaves when profiling short-lived functions over longer running tasks, and how the collected call stacks and timing information evolve across repeated executions. This test set also serves as a practical example suite that others can reuse when validating changes or experimenting with profiling strategies

In addition, the project explicitly documents how these tests reveal the overhead and practical limitations of using PyInstrument in a test-driven workflow. By comparing the execution of the same scenarios with and without profiling, and by repeating the tests several times, it becomes clearer which portion of the total runtime is attributable to the application code and which portion is introduced by the profiler itself. This clarification improves the interpretability of the profiling reports generated by the test suite, and makes it easier to explain why small performance differences must be considered with care when the profiler is active. Together, these test-focused improvements make the repository a more useful resource for understanding how PyInstrument behaves in concrete, code-level examples and for integrating profiling into automated testing practices.

We also resolved several code smells and improved the readability and maintainability of the profiling components. Comments and internal documentation were added to explain how

specific functions interact within the Profiler, making the codebase easier to navigate for new contributors.

Overall, these updates transformed the repository into a more stable and transparent environment for performance evaluation. It now supports more consistent testing, produces more interpretable results, and provides a clearer understanding of how PyInstrument behaves under varying conditions.

Overall Quality Assessment

The project now demonstrates solid robustness for standard synchronous profiling tasks and basic test execution in controlled environments, marking a clear improvement over its initial state where setup inconsistencies often led to unreliable runs. While core functionality handles everyday usage reliably, the remaining risks center on asynchronous code paths, which lack comprehensive coverage, and environment dependent configurations that can still cause intermittent failures across Python versions or OS setups. Compared to the project's start, maintainability has advanced through clearer documentation and structured test scripts, though setup complexity persists as a barrier, making it no simpler for newcomers than before despite added reproducibility aids.

Unit tests emerged as the most effective method for gaining insight into PyInstrument behavior, outperforming exploratory load / stress attempts with tools like Locust by providing consistent, repeatable data on overhead and call stack accuracy. These tests excel at exposing fine grained performance patterns in CPU-bound scenarios, where higher-level stress simulations revealed less actionable detail due to their overhead and variability. Testability has notably increased from the outset with the new suite in `courseProjectDocs` offering targeted scenarios that validate profiler stability far better than the original example.

Overall quality reflects steady progress in reliability and insight generation, yet falls short of production-grade maturity due to unaddressed async gaps and persistent setup friction areas that demand targeted expansion for broader applicability.

Lessons Learned

Jose

Through this project several practical insights emerged that will guide future work with PyInstrument and similar profilers. First, statistical profilers always introduce overhead that can mislead small measurements. Repeated baseline tests without profiling are essential to quantify this effect accurately before drawing conclusions about code performance. Simple unit tests targeting short, repeated functions provided the clearest visibility into tool behavior, far outperforming more complex lead simulations that added unnecessary variability. Clear step by

step environment setup documentation proved critical for reproducibility as even minor Python version or dependency differences caused inconsistent results. Finally focusing on synthetic but realistic test scenarios balanced effort and insight effectively, revealing profiler limitations without over engineering the validation suite. These lessons strengthen my approach to performance analysis in academic and development projects moving forward.

Michael

Throughout the project, I learned a great deal about working as a team, along with dealing with new repositories and projects. With my time transitioning between two different projects, I realized how well and how essential good documentation is the cornerstone of any good project, big or small. By making sure that you have good comments and organization of files, it can make it easier and simpler for people to understand your project. Not only that, but people can have an easier time collaborating with other programmers. Not only that, I learned that I can rely on my teammates more when it comes to asking for assistance on tasks. Being in a space where everyone knows each other and their craft; being able to learn from each other is essential.

Ursula

Through this project, I saw firsthand how profiling can reveal performance bottlenecks that are not visible through functional testing alone. I also gained insight into the importance of testing software across multiple environments, particularly on systems with known compatibility considerations such as Apple Silicon Macs, where behavior can differ in subtle but impactful ways. Additionally, I learned about a variety of testing and analysis tools that I can apply to my own software projects to improve reliability, performance, and overall code quality!

I also genuinely enjoyed working with my team on this project! Collaborating on a real-world tool and testing it from the ground up across multiple phases made the experience really engaging and rewarding. Being able to apply different testing techniques in a structured way, while sharing responsibility and insights with my teammates, made the project especially fun. Overall, this was a really cool class, and the project helped reinforce both technical skills and teamwork in a meaningful way!

Team Members and Roles

Overall, work on the project was divided as evenly as possible among us, with tasks generally split into thirds. Responsibilities such as tool setup and debugging were rotated based on individual weekly schedules and backgrounds/interests.

Jose

For the unit testing assignment, he worked on the 5 profiler tests. For mutation testing, he did the setup of the mutmut library and the “context change frame handling” test. For static analysis, he set up the static analysis tool Ruff, fixed simple issues using `ruff check . --fix`, and manually address errors “E731” and “E722.” For integration testing, he did the implementation of “test integration profiler.” For system testing, he did test case “TC-E2E-001” and documented the results. For security testing, he did vulnerabilities “3” and “4” in the summary table. For performance testing, he did the tool setup, and performed and documented the load test. For this document, he worked on the “Improvements Made to the Project” and “Overall Quality Assessment” sections, as well as his “Lessons Learned” section.

Michael

For the unit testing assignment, he worked on the 5-frame tests. For mutation testing, he assisted with general testing along with doing the “shorten path equal depth” test. For static analysis, he tackled a bug within E731, changing the formatting of the “show_pyinstrument” function. For integration testing, he did the “Test profiler” and “jsonRenderer” tests. For system testing, he designed and executed the JSON UI end-to-end system test. For security testing, he did vulnerability “5” in the summary table. For performance testing, he performed and documented the spike test. For this document, he worked on the “Most Significant Issues Found” supersection, as well as his “Lessons Learned” section.

Ursula

For the unit testing assignment, she worked on the 5 edge case tests. For mutation testing, she helped with setup debugging and did the “shorten path” test. For static analysis, she did the F841 and E712 fixes. For integration testing, she did the “test profiler” and “html” tests. For system testing, she did test case “TC-E2E-001” and wrote the error report describing how the `-o` flag did not create an output file on her macbook, along with some theories as to why. For security testing, she did the tool setup, and vulnerabilities “1” and “2” in the summary table. For performance testing, she performed and documented the stress test. For this document, she worked on the “Highlights of Each Testing Stage” supersection, “Project Overview,” “Team Members and Roles,” and her “Lessons Learned” section.

References

Grammarly for spelling and grammar assistance

ChatGPT for help with tone and phrasing

Out github repo: <https://github.com/ucp4496/pyinstrument>

The Pyinstrument project: <https://github.com/joerick/pyinstrument>